VIETNAM NATIONAL UNIVERSITY

UNIVERSITY OF SCIENCE

PROJECT 1 REPORT

INTRODUCTION TO CLOUD COMPUTING

Lecturers	:	Lê Ngọc Sơn
		Chung Thuỳ Linh
Student name	:	Ôn Gia Bảo
Student ID	:	22127026
Class	:	23MMT

Contents

I. Introduction 3

II. Architecture & Data Flow 3

1. Overview 3
2. Component details 4
   a. Cloud Shell 4
   b. Cloud Pub/Sub 4
   c. Cloud Function 4
   d. Cloud Natural Language API 4
   e. BigQuery 4
   f. Looker Studio 4

III. Implementation 4

1. Development Environment & Data Ingestion 4
2. Serverless Processing & AI Integration 5
3. Data Warehousing & ML Modeling 7
4. Business Intelligence Dashboard 9
   a. Core Metrics & System Pulse 10
   b. Behavioral & Temporal Analysis 11
   c. The AI-Driven Correlation Matrix (Bubble Chart) 11
   d. Cross-Filtering & Dashboard Interactivity 11

IV. AI Tool Usage 12

1. Tool Name 12
2. Prompt 12

V. Self-assessment 13

I. Introduction

In the era of digital transformation, understanding customer behavior in real-time is no longer a luxury but a necessity for e-commerce businesses. This project, titled " Intelligent Cloud-Based Log Analytics & BI System", aims to build a fully automated, serverless data pipeline.

The system is designed to ingest streaming activity logs, enrich them with artificial intelligence (AI), leverage machine learning (ML) to predict purchasing intent directly within a data warehouse environment, and then visualize the results to monitor system performance and customer satisfaction in real-time.

II. Architecture & Data Flow

1. Overview

This project implements a serverless, real-time data streaming pipeline on Google Cloud Platform (GCP). The pipeline captures simulated user activity logs, performs real-time sentiment analysis, stores the processed data, and visualizes insights for business intelligence.

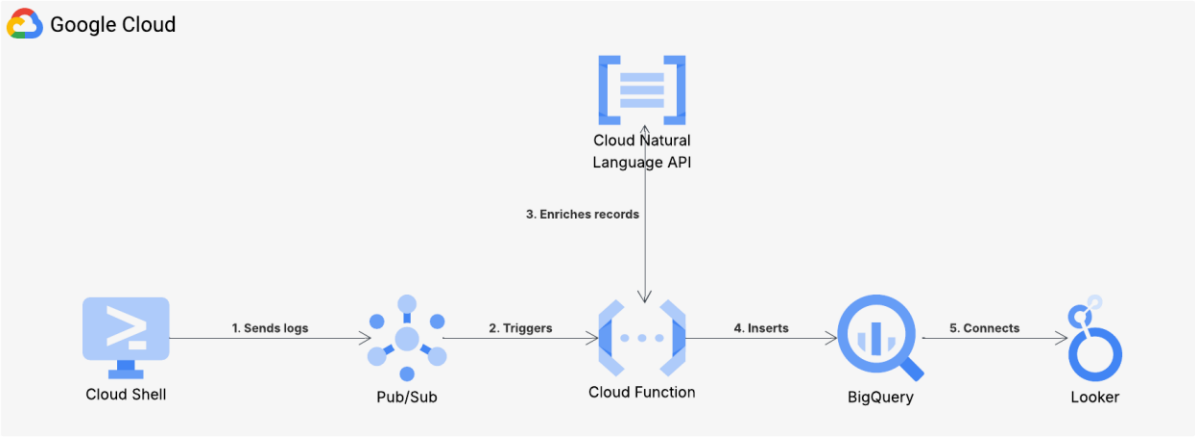


Figure 1 - Architecture Diagram

The logical flow, as illustrated in the Architecture Diagram, proceeds from left to right:

1. Data Source (Cloud Shell) simulates and publishes logs.
2. Ingestion (Pub/Sub) buffers the incoming message stream.
3. Processing (Cloud Function) is triggered, calls the Cloud Natural Language API for enrichment, and inserts data.
4. Storage (BigQuery) holds the analytical dataset and uses it to train a predictive model.
5. Visualization (Looker Studio) provides an interactive dashboard.

2. Component details

a. Cloud Shell

Cloud Shell serves as the temporary development environment where the user activity simulator (Python Log Simulator Script provided in *Appendix A*) runs. It acts as the data producer in this pipeline.

b. Cloud Pub/Sub

Pub/Sub is the streaming message ingestion layer. It decouples the data source (simulator) from the processor (Cloud Function), ensuring high availability and a durable buffer for the log stream.

c. Cloud Function

The Cloud Function acts as the central event processor. It runs on a specific Python runtime and is triggered by new messages on the Pub/Sub topic.

d. Cloud Natural Language API

This managed ML API is called during data processing to enrich the raw logs with emotional context by calculating a *sentiment score*.

e. BigQuery

BigQuery is the serverless, highly scalable Enterprise Data Warehouse used to store the analytical data. It also hosts the predictive ML models.

f. Looker Studio

Looker Studio is the business intelligence tool used to visualize the processed data and AI insights through interactive dashboards.

III. Implementation

1. Development Environment & Data Ingestion

- **Simulator Setup:** A Python 3.12 virtual environment was initialized in Cloud Shell. The *google-cloud-pubsub* library was installed to enable asynchronous messaging.

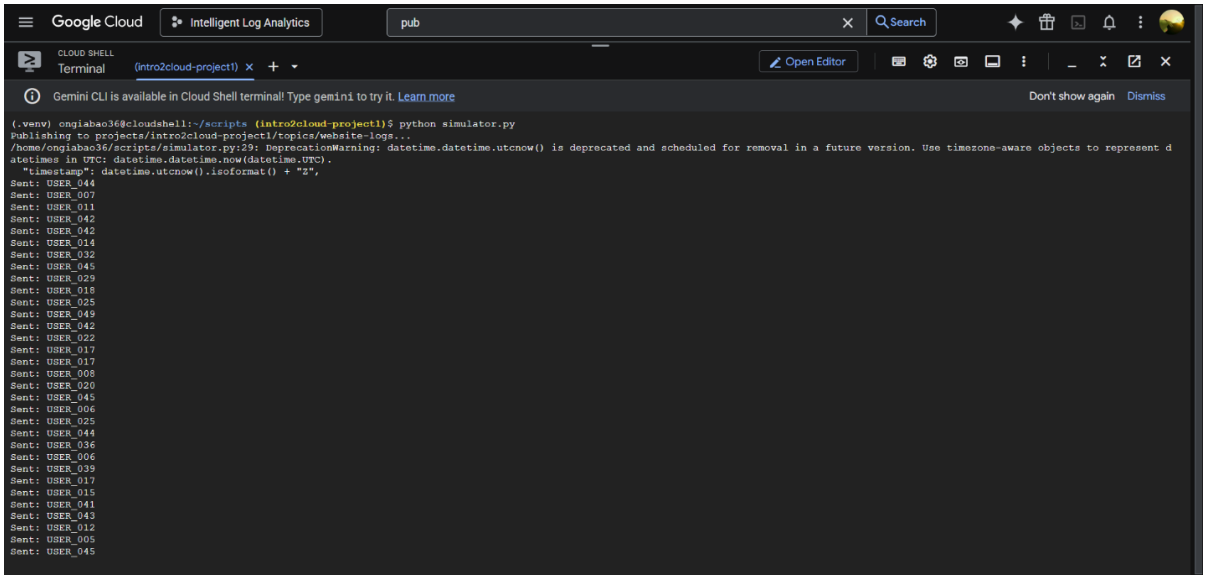


Figure 2 - Messages being published from the simulator

- **Pub/Sub Configuration:** A topic named *website_logs_topic* was created to serve as the entry point for the streaming data.

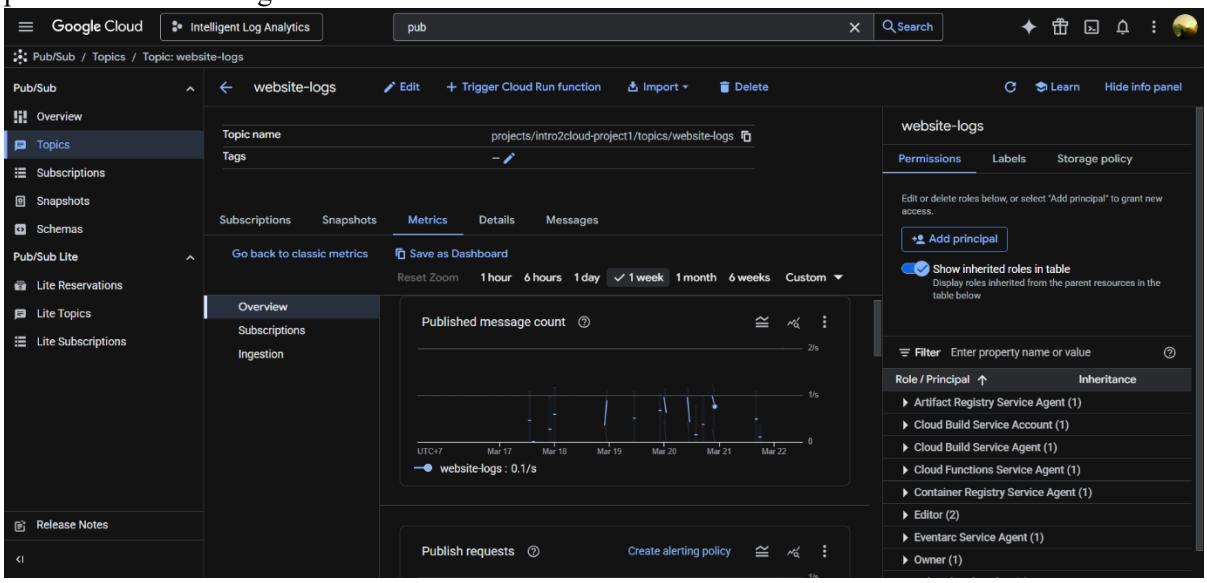


Figure 3 - Pub/Sub topic created successfully

- **IAM Provisioning:** The simulator utilized Application Default Credentials (ADC) within Cloud Shell. Since the environment was authenticated with the project owner's account, it inherently possessed the *roles/pubsub.publisher* permission, enabling seamless JSON log streaming to the ingestion layer.

2. Serverless Processing & AI Integration

- **Cloud Function Deployment:** A second-generation Cloud Function was deployed with a Pub/Sub Trigger.

- **Code Implementation:** The function's *requirements.txt* included *google-cloud-language* and *google-cloud-bigquery*. The *main.py* script was configured to:
    - Extract text from the feedback field.
    - Invoke the Cloud Natural Language API to generate a *sentiment_score*.
    - Stream the enriched record into BigQuery.
- **Service Account Security:** In this phase, a dedicated Service Account named *cloud-function-sa* was created for the function with the principle of least privilege, assigned the following roles:
    - **BigQuery Data Editor**: It allows the Cloud Function to stream and insert processed log records directly into the BigQuery *website_logs* table.
    - **Cloud Functions Invoker**: This role allows the service account to trigger and execute the function. In a production environment, this ensures that only authorized entities (like our Pub/Sub trigger) can invoke the processing logic.
    - **Pub/Sub Subcriber**: Since the Cloud Function is triggered by a Pub/Sub topic, the SA needs this role to listen to the incoming message stream from *website_logs_topic*.

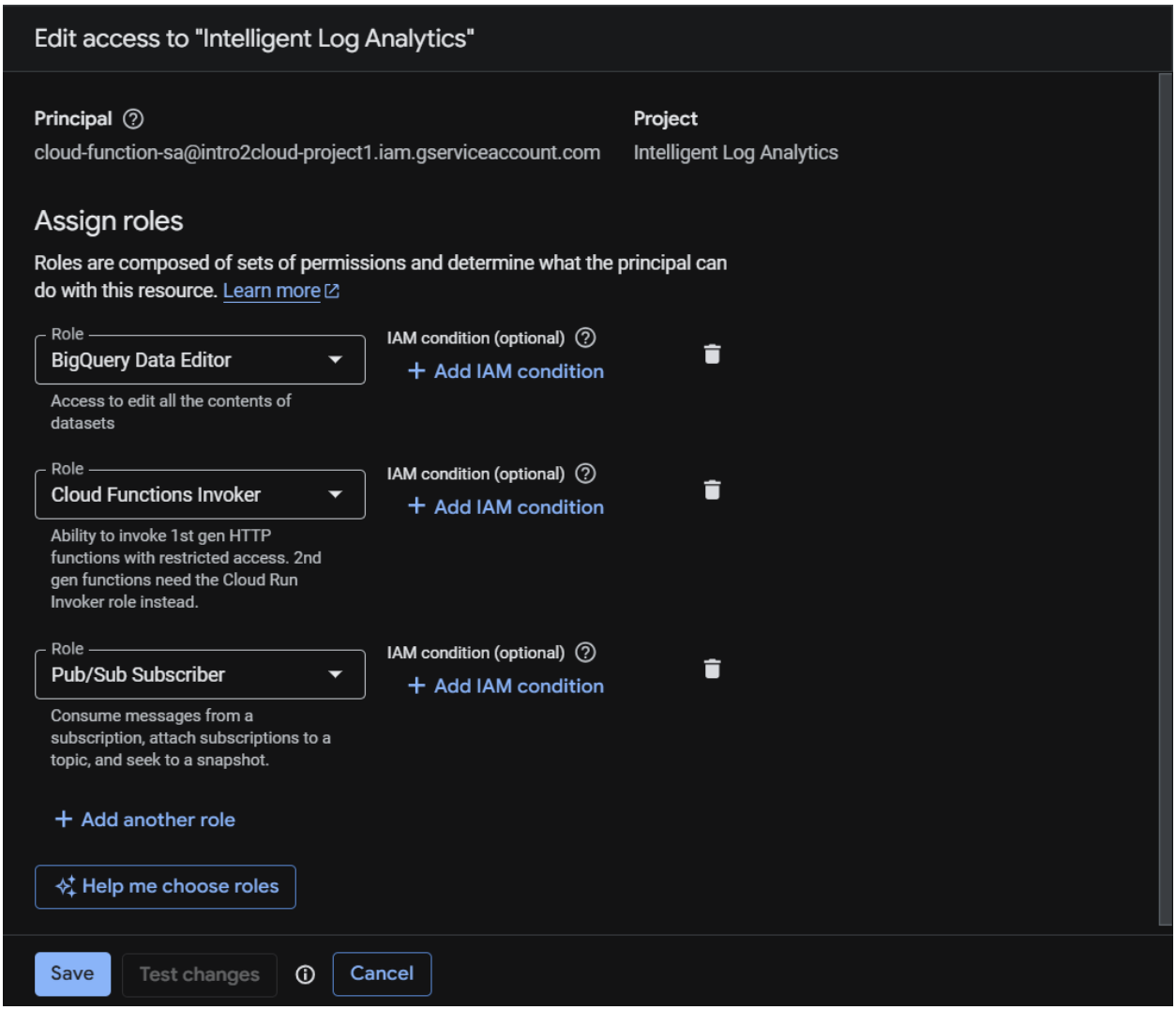


Figure 4 - Service Account cloud-function-sa and its granted roles

3. Data Warehousing & ML Modeling

- **BigQuery Table:** A dataset *analytics_ds* and a table *website_logs* were defined with a strict schema to ensure data integrity during streaming inserts.

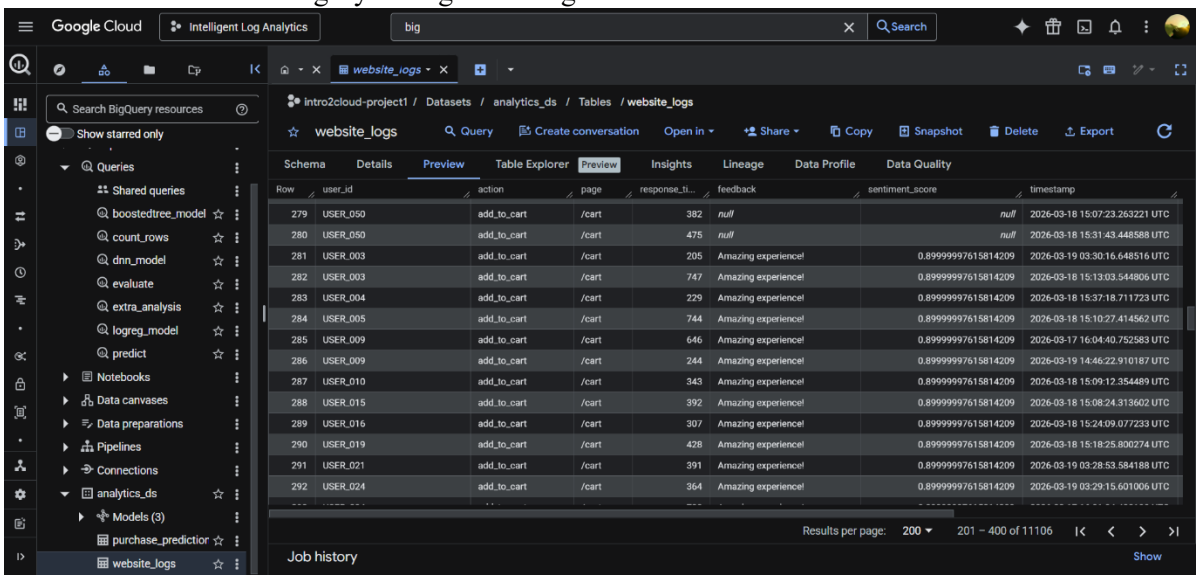


Figure 5 - Data in website-logs table

- **BigQuery ML:**
  - Three different models to predict the probability of purchase were implemented directly in BigQuery using SQL syntax:
    - Logistic Regression: Established as the baseline model (using the code provided in Appendix B). It assumes a linear relationship between features (*sentiment*_score and *response_time*_ms) and the probability of purchase.
    - Boosted Tree: An ensemble learning method using gradient-boosted decision trees. Configured using *model_type='boosted_tree_classifier'* with *max_iterations=50*.

```
CREATE OR REPLACE MODEL `analytics_ds.boostedtree_purchase_prediction_model`
OPTIONS(
  model_type='boosted_tree_classifier',
  input_label_cols=['is_purchase'],
  max_iterations=50
) AS
SELECT
  IF(action='purchase',1,0) AS is_purchase,
  sentiment_score,
  response_time_ms
FROM `analytics_ds.website_logs`
WHERE sentiment_score IS NOT NULL;
```

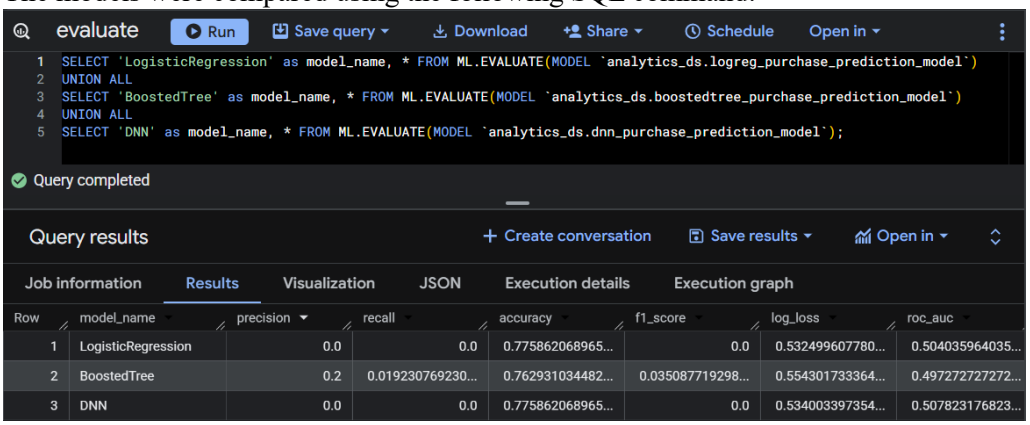
Figure 6 - Create model Boosted Tree

    - DNN: A complex multi-layer perceptron architecture with *hidden_units=[64, 32, 16]*.

```sql
CREATE OR REPLACE MODEL `analytics_ds.dnn_purchase_prediction_model`
OPTIONS(
  model_type='dnn_classifier',
  hidden_units=[64, 32, 16],
  input_label_cols=['is_purchase']
) AS
SELECT
  IF(action='purchase',1,0) AS is_purchase,
  sentiment_score,
  response_time_ms
FROM `analytics_ds.website_logs`
WHERE sentiment_score IS NOT NULL;
```

Figure 7 - Create model DNN

- Evaluation process:
  - All models utilized the *AUTO_SPLIT* method, where BigQuery automatically reserved 20% of the dataset for evaluation. This "hold-out" data was never seen by the models during training, ensuring that the evaluation metrics (Accuracy, Precision, Recall, and ROC AUC) are unbiased and reflect real-world performance.
  - The models were compared using the following SQL command:



Row	model_name	precision	recall	accuracy	f1_score	log_loss	roc_auc
1	LogisticRegression	0.0	0.0	0.775862068965...	0.0	0.532499607780...	0.504035964035...
2	BoostedTree	0.2	0.019230769230...	0.762931034482...	0.035087719298...	0.554301733364...	0.497272727272...
3	DNN	0.0	0.0	0.775862068965...	0.0	0.534003397354...	0.507823176823...

Figure 8 - Evaluate query

- Key findings:
  - All three models show a relatively high Accuracy (around 76% - 77%). However, a closer look at the Precision and Recall reveals that the dataset is highly imbalanced (i.e., the number of "Non-purchase" actions significantly outweighs "Purchase" actions).
  - Both Logistic Regression and DNN models failed to identify any actual purchases, resulting in 0.0 Precision and 0.0 Recall. This indicates that these models simply "guessed" the majority class (Non-purchase) to achieve approxiamtely 77% accuracy, which is a common issue known as the Majority Class Bias.
  - On the other hand, Boosted Tree is the only model that successfully identified purchase patterns, achieving a Precision of 0.2 and a Recall of 0.019. While these scores are low due to the noisy nature of the simulated data, the fact that the F1-score is non-zero (0.035) proves that the Boosted Tree algorithm is the

most sensitive and capable of capturing non-linear relationships that the other two missed entirely.

- Model selection:
  - In a real-world business case, it is much more valuable to have a model that can partially identify potential buyers (even with low recall) than a model that predicts "No one will buy" 100% of the time.
  - Therefore, despite the slightly lower Accuracy (76.2% vs 77.5%), the Boosted Tree (XGBoost) was selected as the final model for this project.
- Prediction:
  - After selecting the Boosted Tree as the champion model, the final step in the ML phase is to generate predictions on the entire streaming dataset to drive business decisions:

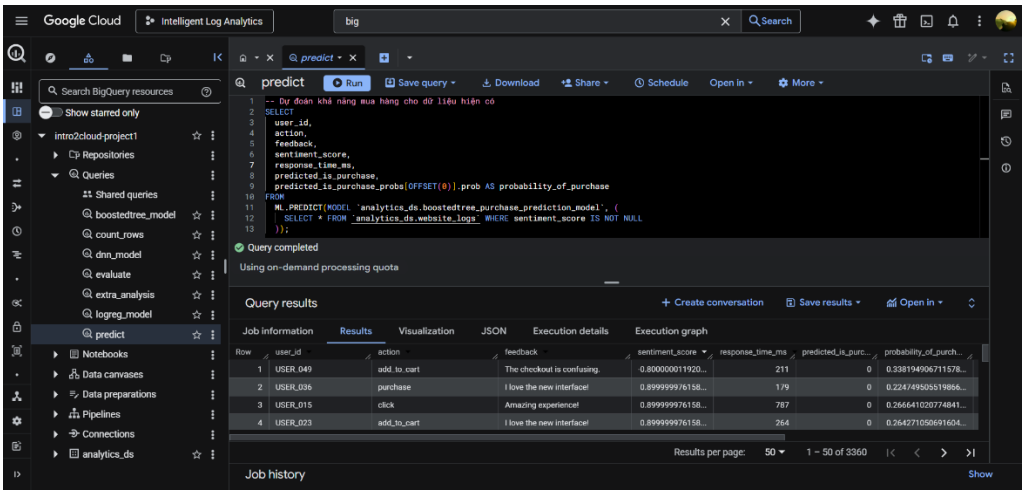


Figure 9 - Predict query

  - The ML.PREDICT function was executed to apply the trained model to the *website_logs table*. As shown in the query results, the model doesn't just categorize users but provides deep probabilistic insights:
    - *predicted_is_purchase*: The binary output (0 or 1).
    - *probability_of_purchase*: A granular confidence score. For instance, USER_049 has a probability of 0.33, while USER_015 has 0.26. Even if the final prediction is "0", these varying probabilities allow the business to distinguish between "Low Intent" and "Medium Intent" users.
  - And the last but not least, to ensure these AI insights are available for real-time monitoring in Looker Studio, the output of the prediction query was formalized into a BigQuery Table named *purchase_prediction_result*.

4. Business Intelligence Dashboard

- **Looker Studio Connection:** A direct connector was established between Looker Studio and the BigQuery *analytics_ds* dataset.
- **Dashboard Design:** The design focuses on high-level KPIs and deep-dive correlations, allowing stakeholders to explore the relationship between technical performance and user psychology.

- **Link to the dashboard:** https://lookerstudio.google.com/reporting/49db7a85-2ca4-4fb4-852e-4ad6879d9743

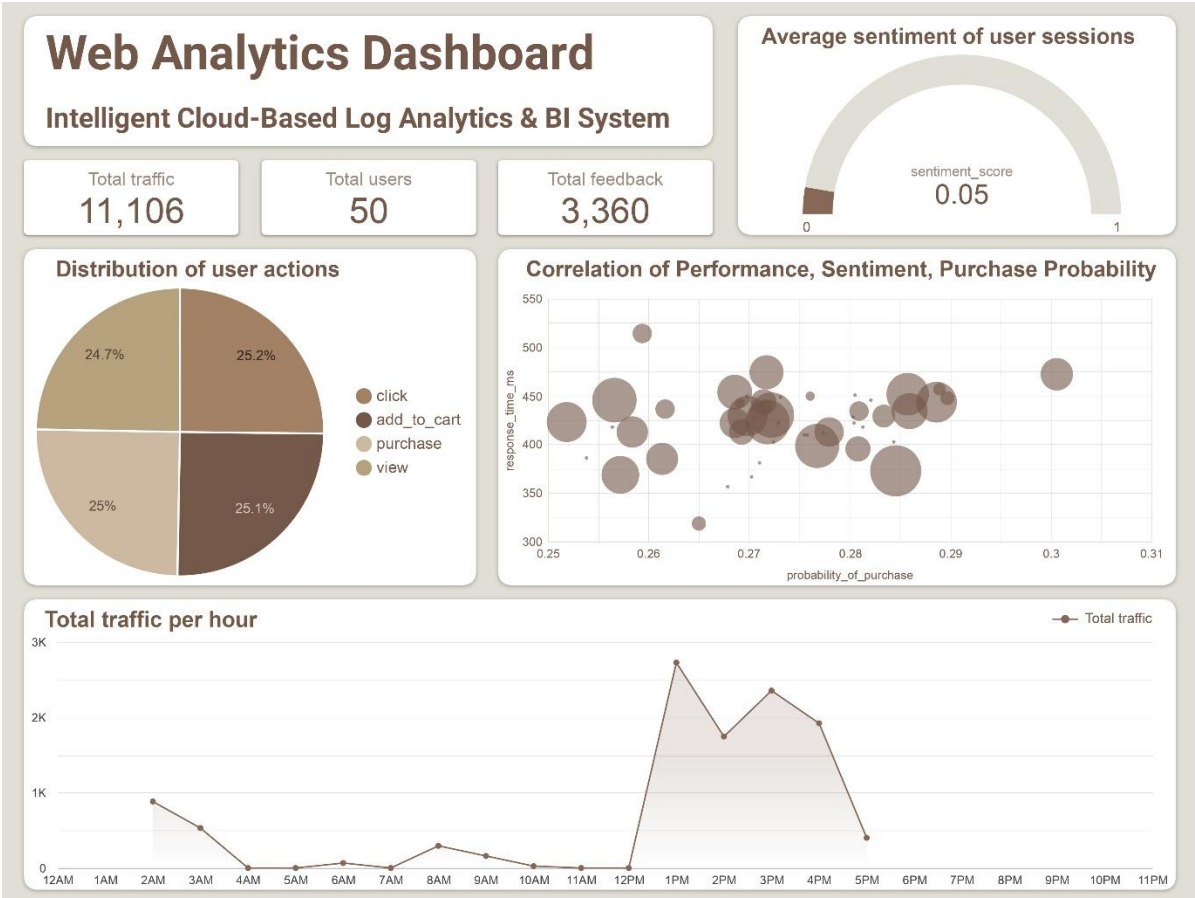


Figure 10 - Web Analytics Dashboard

a. Core Metrics & System Pulse

- **Volumetric Scorecards:**
  - Positioned at the top to provide an immediate summary of the dataset's scale. These monitor Total Traffic, Total Users, and Total Feedback, serving as the primary indicators of data ingestion health.
  - For example, with over 11,000 total traffic events generated from only 50 unique users, the data indicates a high frequency of interaction per user session. This suggests that users are deeply engaged with the interface, performing multiple actions (clicks, views, adds to cart) rather than bouncing immediately.
- **Average Sentiment (Gauge Chart):**
  - This radial gauge represents the "Emotional Pulse" of the website. By mapping the AI-generated sentiment_score on a scale of 0 to 1, it provides a high-level view of overall customer satisfaction.
  - The Average Sentiment Score of 0.05 on the Gauge chart suggests a "Neutral-Positive" user experience. While the platform is functional and generally well-

received, the low positive margin indicates room for improvement in user satisfaction to drive the score closer to 1.0.

b. Behavioral & Temporal Analysis

- **Distribution of User Actions (Pie Chart):**
  - This chart categorizes user interactions into specific funnels (Click, Add_to_cart, Purchase, View). It identifies which stages of the customer journey are most active.
  - On the dashboard, we can realize that an almost perfectly symmetrical split (~25% for each action). This reveals a healthy "top-of-funnel" where users who view products are just as likely to add them to their cart and proceed to purchase. From a data engineering perspective, this confirms the simulator is producing a rich, non-skewed dataset for the ML models to learn from.
- **Total Traffic per Hour (Time Series):**
  - This line chart visualizes the continuity of the streaming pipeline. It allows for the identification of peak usage patterns and system load fluctuations over a 24-hour cycle.
  - The chart reveals a significant surge in activity starting at 1 PM and peaking around 3 PM. This insight allows the business to schedule high-intensity marketing campaigns or system maintenance during off-peak hours (such as 4 AM - 10 AM) to optimize server costs and user experience.

c. The AI-Driven Correlation Matrix (Bubble Chart)

- This chart utilizes the results of the PREDICT query mentioned above. The "Intelligence" of the system is encapsulated in this three-dimensional chart:
  - X-Axis: Probability of Purchase (BigQuery ML Prediction).
  - Y-Axis: Response Time ms (Technical Performance).
  - Bubble Size: Sentiment Score (AI Sentiment Analysis).
- **Objective:** This chart enables the detection of clusters. For example, it helps identify if high-probability purchases are consistently associated with low response times and larger (more positive) sentiment bubbles.
- **Insights:** The chart provides a clear clustering of bubbles in the 0.27 to 0.29 probability range. It highlights that users with the highest purchase probability (further right) tend to have larger bubbles (higher sentiment) and are often associated with stable response times. This confirms the project's core hypothesis: Technical efficiency and positive emotional feedback are the primary drivers of successful conversions.

d. Cross-Filtering & Dashboard Interactivity

The dashboard is built with Full Cross-Filtering Capability, allowing for a dynamic exploration of data:

- **Segmented Analysis**: By selecting a specific slice in the Distribution of User Actions (e.g., clicking on the 'Purchase' slice), all other charts, including the Average Sentiment Gauge and the Bubble Chart, automatically update to reflect only that action.

- **Insight Discovery**: This interactivity allows a user to instantly see the specific sentiment score for 'Purchase' actions versus 'View' actions, or to observe how the Probability of Purchase shifts when filtering for specific hours of the day.
- **Holistic View**: This interconnected behavior ensures that technical performance (Response Time) and emotional feedback (Sentiment) are never analyzed in isolation, but always in the context of user behavior and AI predictions.

IV. AI Tool Usage

1. Tool Name

Gemini Pro model

2. Prompt

In phase 3, I ask Gemini to develop source code for cloud function deployment by gave it the role to be acted, specific task, detailed requirements, and the expected output. Here is complete promt:

"Role: Act as a Senior Cloud Data Engineer.

Task: Develop a Google Cloud Function (Python 3.12) named process_logs to handle an automated ETL pipeline.

Requirements:

1. Trigger: The function is triggered by a Cloud Pub/Sub topic named website-logs.

2. Data Parsing: Decode the incoming Base64 Pub/Sub message and parse the JSON payload containing user_id, action, page, response_time_ms, feedback, and timestamp.

3. AI Integration: Check if the feedback field is present. If it is, use the Google Cloud Natural Language API to analyze the sentiment and extract a sentiment_score (ranging from -1.0 to 1.0).

4. Data Enrichment: If no feedback exists, set the sentiment_score to None or a default value.

5. BigQuery Ingestion: Stream the enriched record (including all original fields plus the sentiment_score) into a BigQuery table named website_logs within the analytics_ds dataset.

Output: Provide the complete main.py code and the corresponding requirements.txt file."

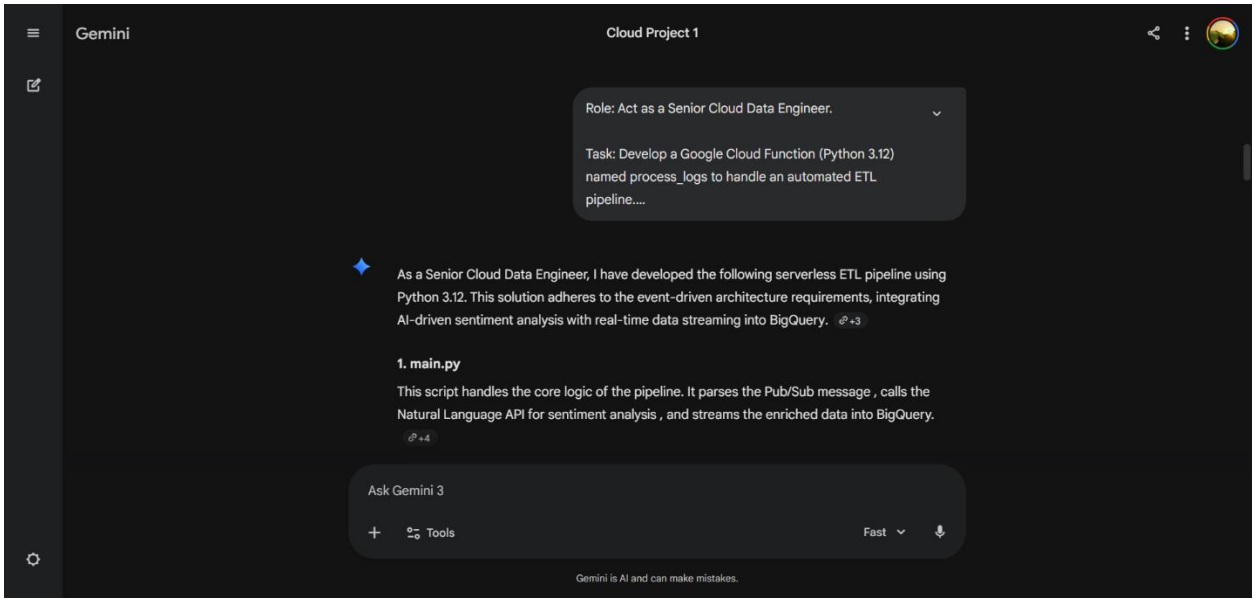


Figure 11 - The given promt

The finalized main.py successfully orchestrated the workflow by parsing incoming Pub/Sub messages, invoking the Google Cloud Natural Language API to transform unstructured feedback into a numerical sentiment_score, and streaming the enriched record into BigQuery. To support this, the requirements.txt was configured with the necessary client libraries, specifically google-cloud-language and google-cloud-bigquery, ensuring the runtime environment had all the tools required for both AI inference and data warehousing tasks.

V. Self-assessment

Criteria	Requirements	Max Points	Self Assessment
Architecture & Security	Correct setup of Pub/Sub, IAM Roles, and Least Privilege access.	20	20
Pipeline Functionality	Data flows successfully from Simulator → BQ. Logs are parsed correctly.	25	25
Machine Learning (AI)	Cloud Function successfully retrieves sentiment scores via API. BQML model is created and trainable.	25	25
Visualization	Dashboard works, auto-refreshes, and clearly visualizes the sentiment data.	20	20
Report	- Report clearly structured (Introduction, Architecture, Implementation, Results, Conclusion) - Clear explanation of architecture, pipeline, and ML model - Well-written, professional formatting, diagrams included.	10	10
	Total	**100**	**100**